Soccer Management System – Phase 1 Domain Design

1. Domain Description

The Soccer Management System manages real soccer data including leagues, teams, stadium locations, and matches. The system allows administrators and users to organize teams into leagues, manage player rosters, schedule matches, verify stadium availability, and track match results. The platform is designed so it can integrate with external soccer APIs to retrieve real-world data about leagues, teams, fixtures, and venues.

2. Subdomain Identification

Core / Orchestration Subdomain

Match Management: Coordinates the main workflow of the system. It schedules matches, verifies teams and stadiums, and records match results. It also aggregates information coming from the supporting modules.

Supporting Subdomains

League Management: Manages soccer leagues and seasons such as the English Premier League, La Liga, MLS, or other professional competitions. It keeps track of which teams participate in each league and the rules of the competition.

Team Management: Manages team information including real clubs such as Arsenal, Real Madrid, or Toronto FC. It stores player rosters, coaching staff information, and team eligibility for matches.

Location Management: Manages stadium and venue data such as Emirates Stadium, Camp Nou, or BMO Field. It keeps track of stadium capacity, location, and availability for hosting matches.

3. Detailed Domain Modeling

3.1 Match Management (Core Subdomain)

Aggregate 1: Match (Aggregate Root)
Entities: MatchEvent, ScoreEntry
Value Objects: MatchTime, MatchResult

Key Attributes:
Match: matchId (UUID), leagueId (UUID), homeTeamId (UUID), awayTeamId (UUID),

stadiumId (UUID), matchTime (MatchTime), status (MatchStatus)
MatchEvent: eventId (UUID), minute (int), eventType (String), teamId (UUID), playerName (String)
ScoreEntry: homeGoals (int), awayGoals (int)
MatchTime: startDateTime (LocalDateTime)
MatchResult: winnerTeamId (UUID), summary (String)

Example Real Data:
League: Premier League
Home Team: Arsenal
Away Team: Chelsea
Stadium: Emirates Stadium

Invariant:
A match must always involve two different teams. A match cannot be marked as COMPLETED unless a score has been recorded.

Aggregate 2: MatchSchedule (Aggregate Root)
Entities: ScheduledMatchRef
Value Objects: ScheduleWindow

Key Attributes:
MatchSchedule: scheduleId (UUID), leagueId (UUID), season (String), status (String), scheduleWindow (ScheduleWindow)
ScheduledMatchRef: matchId (UUID), roundNumber (int)
ScheduleWindow: startDate (LocalDate), endDate (LocalDate)

Invariant:
A schedule cannot contain duplicate matches in the same round. A match must be scheduled within the allowed season window.

3.2 League Management (Supporting Subdomain)

Aggregate Root: League
Entities: SeasonRegistration
Value Objects: LeagueRules

Key Attributes:
League: leagueId (UUID), name (String), country (String), division (String), active (Boolean)
SeasonRegistration: registrationId (UUID), teamId (UUID), season (String), registeredAt (LocalDate)
LeagueRules: maxTeams (int), pointsForWin (int), pointsForDraw (int)

Example Real Data:
League: English Premier League

Country: England
Max Teams: 20

Invariant:
A league cannot register more teams than the maximum allowed by its rules. An inactive league cannot accept new team registrations.

3.3 Team Management (Supporting Subdomain)

Aggregate Root: Team
Entities: Player
Value Objects: TeamName, CoachInfo

Key Attributes:
Team: teamId (UUID), teamName (TeamName), foundedYear (int), status (String), coachInfo (CoachInfo)
Player: playerId (UUID), fullName (String), jerseyNumber (int), position (String), active (Boolean)
TeamName: shortName (String), fullName (String)
CoachInfo: coachName (String), licenseLevel (String)

Example Real Data:
Team: Arsenal FC
Coach: Mikel Arteta
Player Example: Bukayo Saka – Right Wing

Invariant:
A team must have at least 11 active players to participate in official matches.

3.4 Location Management (Supporting Subdomain)

Aggregate Root: Stadium
Entities: MaintenanceWindow
Value Objects: Address, FieldDimensions

Key Attributes:
Stadium: stadiumId (UUID), name (String), capacity (int), active (Boolean), address (Address), fieldDimensions (FieldDimensions)
MaintenanceWindow: maintenanceWindowId (UUID), startDateTime (LocalDateTime), endDateTime (LocalDateTime), reason (String)
Address: street (String), city (String), country (String)
FieldDimensions: lengthMeters (double), widthMeters (double)

Example Real Data:
Stadium: Emirates Stadium
City: London

Capacity: 60,000

Invariant:
A stadium cannot host a match if it is inactive or during a maintenance period.

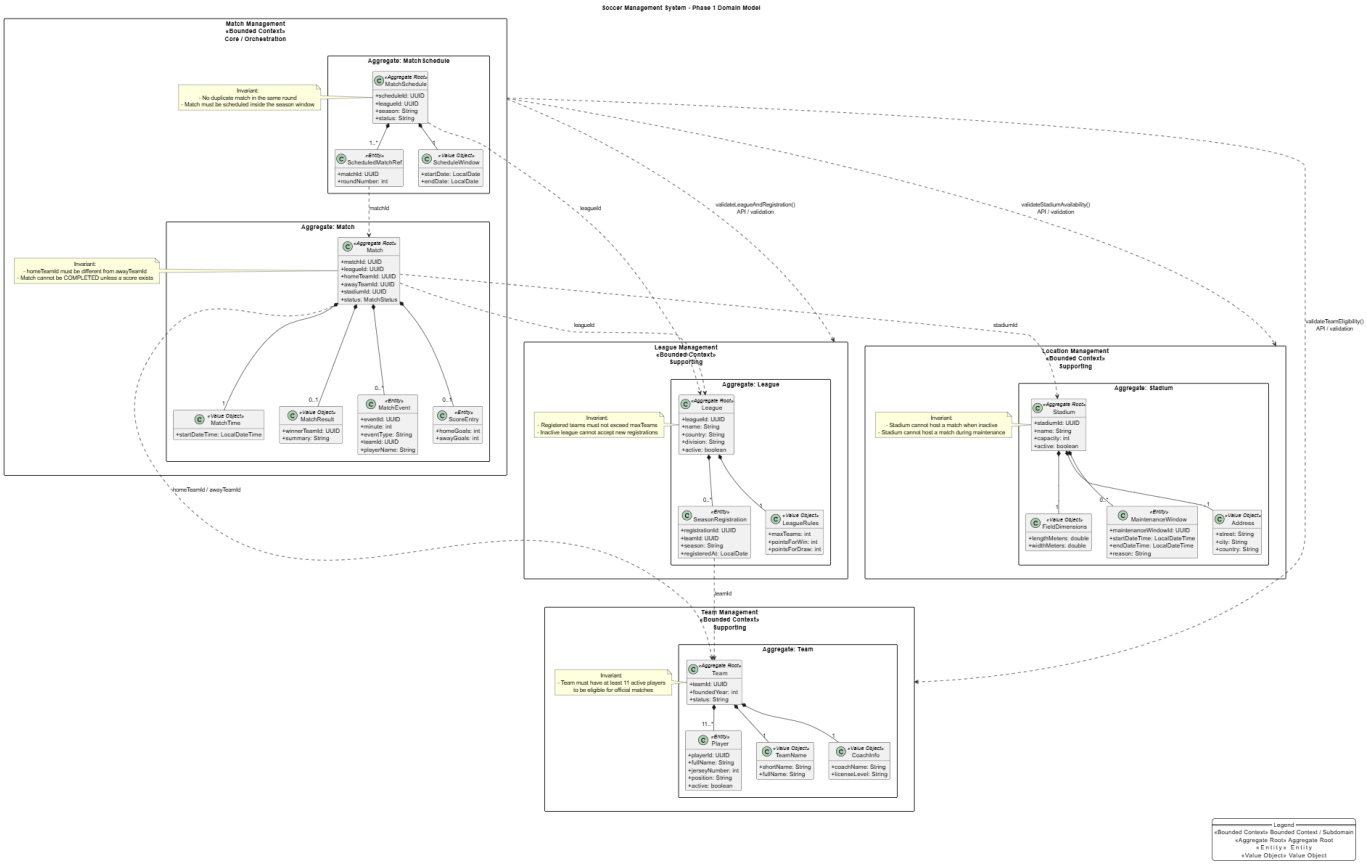
4. Relationship Summary

The Match Management subdomain coordinates the main workflow of the system. When a match is created or scheduled, it verifies that the league exists and that both teams are registered within that league. It then checks that both teams are eligible to play and verifies that the selected stadium is available at the scheduled time. These validations are performed by interacting with the League Management, Team Management, and Location Management subdomains using references by ID.

5. Integration with External Soccer APIs

The system is designed to integrate with external soccer data APIs such as API-Football, TheSportsDB, or football-data.org. These APIs provide real data about leagues, teams, players, fixtures, and stadiums. The integration would be implemented in the infrastructure layer of each module using API clients that retrieve and map external data into the application models.

6. Design Decisions

This design follows Domain Driven Design principles. Match Management acts as the core orchestration module because it coordinates the main business workflow. League Management, Team Management, and Location Management each focus on a specific domain responsibility and support the core module. The system is structured as a modular monolith so that all modules remain within one Spring Boot application while still maintaining clear boundaries between domain contexts.

PLANT UML :
Match Management is the core bounded context and orchestrates the main workflow.
League Management provides league and registration validation.
Team Management provides team eligibility validation.
Location Management provides stadium availability validation.
All cross-context interactions use ID references and validation calls, not shared entities.